

3

Thread Synchronization Utilities

In this chapter, we will cover the following topics:

- Controlling concurrent access to one or more copies of a resource
- Waiting for multiple concurrent events
- Synchronizing tasks at a common point
- Running concurrent-phased tasks
- Controlling phase change in concurrent-phased tasks
- Exchanging data between concurrent tasks
- Completing and linking tasks asynchronously

Introduction

In *Chapter 2, Basic Thread Synchronization*, you learned the concepts of synchronization and critical sections. Basically, we talk about synchronization when more than one concurrent task shares a resource, for example, an object or an attribute of an object. The blocks of code that access this shared resource are called critical sections.

If you don't use appropriate mechanisms, you might have incorrect results, data inconsistencies, or error conditions. Therefore, we have to adopt one of the synchronization mechanisms provided by the Java language to avoid these problems.

Chapter 2, *Basic Thread Synchronization*, taught you about the following basic synchronization mechanisms:

- The `synchronized` keyword
- The `Lock` interface and its implementation classes: `ReentrantLock`, `ReentrantReadWriteLock.ReadLock`, and `ReentrantReadWriteLock.WriteLock`
- The `StampedLock` class

In this chapter, you will learn how to use high-level mechanisms to synchronize multiple threads. These high-level mechanisms are as follows:

- **Semaphores:** A semaphore is a counter that controls access to one or more shared resources. This mechanism is one of the basic tools of concurrent programming and is provided by most programming languages.
- **CountDownLatch:** The `CountDownLatch` class is a mechanism provided by the Java language that allows a thread to wait for the finalization of multiple operations.
- **CyclicBarrier:** The `CyclicBarrier` class is another mechanism provided by the Java language that allows the synchronization of multiple threads at a common point.
- **Phaser:** The `Phaser` class is another mechanism provided by the Java language that controls the execution of concurrent tasks divided in phases. All the threads must finish one phase before they can continue with the next one.
- **Exchanger:** The `Exchanger` class is another mechanism provided by the Java language that provides a point of data interchange between two threads.
- **CompletableFuture:** The `CompletableFuture` class provides a mechanism where one or more tasks can wait for the finalization of another task that will be explicitly completed in an asynchronous way in future. This class was introduced in Java 8 and has introduced new methods in Java 9.

Semaphores are generic synchronization mechanisms that you can use to protect any critical section in any problem. Other mechanisms are thought to be used in applications with specific features, as described previously. Be sure to select the appropriate mechanism according to the characteristics of your application.

This chapter presents seven recipes that will show you how to use the mechanisms described.

Controlling concurrent access to one or more copies of a resource

In this recipe, you will learn how to use the semaphore mechanism provided by the Java language. A semaphore is a counter that protects access to one or more shared resources.



The concept of a semaphore was introduced by Edsger Dijkstra in 1965 and was used for the first time in the THEOS operating system.

When a thread wants to access one of the shared resources, it must first acquire the semaphore. If the internal counter of the semaphore is greater than 0, the semaphore decrements the counter and allows access to the shared resource. A counter bigger than 0 implies that there are free resources that can be used, so the thread can access and use one of them.

Otherwise, if the counter is 0, the semaphore puts the thread to sleep until the counter is greater than 0. A value of 0 in the counter means all the shared resources are used by other threads, so the thread that wants to use one of them must wait until one is free.

When the thread has finished using the shared resource, it must release the semaphore so that another thread can access the resource. This operation increases the internal counter of the semaphore.

In this recipe, you will learn how to use the `Semaphore` class to protect more than one copy of a resource. You are going to implement an example, which has a print queue that could print documents in three different printers.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `PrintQueue` that will implement the print queue:

```
public class PrintQueue {
```

2. This class will have three private attributes. A semaphore named `semaphore`, an array of Booleans named `freePrinters`, and a lock named `lockPrinters`, as shown in the following code snippet:

```
    private final Semaphore semaphore;  
    private final boolean freePrinters[];  
    private final Lock lockPrinters;
```

3. Implement the constructor of the class. It initializes the three attributes of the class, as shown in the following code snippet:

```
    public PrintQueue() {  
        semaphore=new Semaphore(3);  
        freePrinters=new boolean[3];  
        for (int i=0; i<3; i++){  
            freePrinters[i]=true;  
        }  
        lockPrinters=new ReentrantLock();  
    }
```

4. Implement the `printJob()` method that will simulate the printing of a document. It receives an object called `document` as a parameter:

```
    public void printJob (Object document){
```

5. First of all, the `printJob()` method calls the `acquire()` method to acquire access to the semaphore. As this method can throw an `InterruptedException` exception, you must include the code to process it:

```
        try {  
            semaphore.acquire();
```

6. Then, get the number of the printers assigned to print this job, using the private method `getPrinter()`:

```
        int assignedPrinter=getPrinter();
```

7. Then, implement the lines that simulate the printing of a document waiting for a random period of time:

```
long duration=(long) (Math.random()*10);
System.out.printf("%s - %s: PrintQueue: Printing a Job in
                  Printer %d during %d seconds\n",
                  new Date(), Thread.currentThread().getName(),
                  assignedPrinter,duration);
TimeUnit.SECONDS.sleep(duration);
```

8. Finally, release the semaphore by calling the `release()` method and marking the printer used as free, and assign `true` to the corresponding index in the `freePrinters` array:

```
    freePrinters[assignedPrinter]=true;
} catch (InterruptedException e) {
    e.printStackTrace();
} finally {
    semaphore.release();
}
```

9. Next, implement the `getPrinter()` method. It's a private method that returns an `int` value and has no parameters:

```
private int getPrinter() {
```

10. First, declare an `int` variable to store the index of the printer:

```
    int ret=-1;
```

11. Then, get access to the `lockPrinters` object:

```
    try {
        lockPrinters.lock();
```

12. Post this, find the first `true` value in the `freePrinters` array and save its index in a variable. Modify this value to `false` because this printer will be busy:

```
    for (int i=0; i<freePrinters.length; i++) {
        if (freePrinters[i]){
            ret=i;
            freePrinters[i]=false;
            break;
        }
    }
}
```

13. Finally, free the `lockPrinters` object and return the index of the true value:

```
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        lockPrinters.unlock();
    }
    return ret;
}
```

14. Next, create a class called `Job` and specify that it implements the `Runnable` interface. This class implements the job that will send a document to the printer:

```
public class Job implements Runnable {
```

15. Declare a `PrintQueue` object. Call it `printQueue`:

```
    private PrintQueue printQueue;
```

16. Implement the constructor of the class. It initializes the `PrintQueue` object declared in the class:

```
    public Job(PrintQueue printQueue) {
        this.printQueue=printQueue;
    }
}
```

17. Implement the `run()` method:

```
    @Override
    public void run() {
```

18. First, this method writes a message to the console that shows that the job has started its execution:

```
        System.out.printf("%s: Going to print a job\n",
                           Thread.currentThread().getName());
```

19. Then, it calls the `printJob()` method of the `PrintQueue` object:

```
        printQueue.printJob(new Object());
```

20. Finally, the method writes a message to the console that shows that it has finished its execution:

```
        System.out.printf("%s: The document has been printed\n",
                           Thread.currentThread().getName());
    }
}
```

21. Next, implement the main class of the example by creating a class named `Main` and implementing the `main()` method:

```
public class Main {  
    public static void main (String args[]){
```

22. Create a `PrintQueue` object named `printQueue`:

```
PrintQueue printQueue=new PrintQueue();
```

23. Create 12 threads. Each one of these threads will execute a `Job` object that will send a document to the print queue:

```
Thread[] threads=new Thread[12];  
for (int i=0; i < threads.length; i++){  
    thread[i]=new Thread(new Job(printQueue), "Thread"+i);  
}
```

24. Finally, start the 12 threads:

```
for (int i=0; i < threads.length; i++){  
    thread[i].start();  
}
```

How it works...

The key to this example is the `printJob()` method of the `PrintQueue` class. This method shows three steps you must follow when you use a semaphore to implement a critical section and protect access to a shared resource:

1. First, you acquire the semaphore with the `acquire()` method.
2. Then, you do the necessary operations with the shared resource.
3. Finally, release the semaphore with the `release()` method.

Another important point in this example is the constructor of the `PrintQueue` class and the initialization of the `Semaphore` object. You pass the value 3 as the parameter of this constructor, so you are creating a semaphore that will protect three resources. The first three threads that call the `acquire()` method will get access to the critical section of this example, while the rest will be blocked. When a thread finishes a critical section and releases the semaphore, another thread will acquire it.

The following screenshot shows the output of an execution of this example:

```
Mon Sep 05 00:07:27 GMT+01:00 2016 - Thread 0: PrintQueue: Printing a Job in Printer 0 during 8 seconds
Thread 4: Going to print a job
Thread 3: Going to print a job
Thread 2: Going to print a job
Thread 1: Going to print a job
Mon Sep 05 00:07:27 GMT+01:00 2016 - Thread 8: PrintQueue: Printing a Job in Printer 2 during 6 seconds
Mon Sep 05 00:07:27 GMT+01:00 2016 - Thread 10: PrintQueue: Printing a Job in Printer 1 during 7 seconds
Thread 8: The document has been printed
Mon Sep 05 00:07:33 GMT+01:00 2016 - Thread 6: PrintQueue: Printing a Job in Printer 2 during 8 seconds
Thread 10: The document has been printed
Mon Sep 05 00:07:34 GMT+01:00 2016 - Thread 9: PrintQueue: Printing a Job in Printer 1 during 5 seconds
Thread 0: The document has been printed
Mon Sep 05 00:07:35 GMT+01:00 2016 - Thread 5: PrintQueue: Printing a Job in Printer 0 during 1 seconds
```

You can see how the first three print jobs start at the same time. Then, when one printer finishes its job, another one begins.

There's more...

The `Semaphore` class has three additional versions of the `acquire()` method:

- `acquireUninterruptibly()`: The `acquire()` method, when the internal counter of the semaphore is 0, blocks the thread until the semaphore is released. During this period, the thread may be interrupted; if this happens, the method will throw an `InterruptedException` exception. This version of the `acquire` operation ignores the interruption of the thread and doesn't throw any exceptions.
- `tryAcquire()`: This method tries to acquire the semaphore. If it can, it returns the `true` value. But if it can't, it returns `false` instead of being blocked and waits for the release of the semaphore. It's your responsibility to take correct action based on the return value.
- `tryAcquire(long timeout, TimeUnit unit)`: This method is equivalent to the previous one, but it waits for the semaphore for the period of time specified in the parameters. If the period of time ends and the method hasn't acquired the semaphore, it will return `false`.

The `acquire()`, `acquireUninterruptibly()`, `tryAcquire()`, and `release()` methods have an additional version, which has an `int` parameter. This parameter represents the number of permits the thread that uses them wants to acquire or release, in other words, the number of units that this thread wants to delete or add to the internal counter of the semaphore.

In the case of the `acquire()`, `acquireUninterruptibly()`, and `tryAcquire()` methods, if the value of the counter is less than the number passed as parameter value, the thread will be blocked until the counter gets the same value or a greater one.

Fairness in semaphores

The concept of fairness is used by the Java language in all classes that can have various threads blocked and are waiting for the release of a synchronization resource (for example, a semaphore). The default mode is called **non-fair mode**. In this mode, when the synchronization resource is released, one of the waiting threads is selected and is given this resource; however, it's selected without any criteria. Fair mode, on the other hand, changes this behavior and selects the thread that has been waiting for the longest period of time.

As it occurs with other classes, the `Semaphore` class admits a second parameter in its constructor. This parameter must take a Boolean value. If you give it a false value, you are creating a semaphore that will work in non-fair mode. You will get the same behavior if you don't use this parameter. If you give it a true value, you are creating a semaphore that will work in fair mode.

See also

- The *Monitoring a Lock interface* recipe in Chapter 9, *Testing Concurrent Applications*
- The *Synchronizing a block of code with a lock* recipe in Chapter 2, *Basic Thread Synchronization*

Waiting for multiple concurrent events

The Java concurrency API provides a class that allows one or more threads to wait until a set of operations are made. It's called the `CountDownLatch` class. This class is initialized with an integer number, which is the number of operations the threads are going to wait for. When a thread wants to wait for the execution of these operations, it uses the `await()` method. This method puts the thread to sleep until the operations are completed. When one of these operations finishes, it uses the `countDown()` method to decrement the internal counter of the `CountDownLatch` class. When the counter arrives at 0, the class wakes up all the threads that were sleeping in the `await()` method.

In this recipe, you will learn how to use the `CountDownLatch` class to implement a video conference system. The video conference system should wait for the arrival of all the participants before it begins.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Videoconference` and specify that it implements the `Runnable` interface. This class will implement the video conference system:

```
public class Videoconference implements Runnable{
```

2. Declare a `CountDownLatch` object named `controller`:

```
private final CountDownLatch controller;
```

3. Implement the constructor of the class that initializes the `CountDownLatch` attribute. The `Videoconference` class will wait for the arrival of the number of participants received as a parameter:

```
public Videoconference(int number) {  
    controller=new CountDownLatch(number);  
}
```

4. Implement the `arrive()` method. This method will be called each time a participant arrives for the video conference. It receives a `String` type named `name` as the parameter:

```
public void arrive(String name){
```

5. First, it writes a message with the parameter it has received:

```
System.out.printf("%s has arrived.",name);
```

6. Then, it calls the `countDown()` method of the `CountDownLatch` object:

```
controller.countDown();
```

7. Finally, it writes another message with the number of participants whose arrival is pending, using the `getCount()` method of the `CountDownLatch` object:

```
System.out.printf("VideoConference: Waiting for %d\n", controller.getCount());
```

8. Next, implement the main method of the video conference system. It's the `run()` method that every `Runnable` object must have:

```
@Override
public void run() {
```

9. First, use the `getCount()` method to write a message with the number of participants in the video conference:

```
System.out.printf("VideoConference: Initialization: %d\n", controller.getCount());
```

10. Then, use the `await()` method to wait for all the participants. As this method can throw an `InterruptedException` exception, you must include the code to process it:

```
try {
    controller.await();
```

11. Finally, write a message to indicate that all the participants have arrived:

```
System.out.printf("VideoConference: All the participants have\n\n");
System.out.printf("VideoConference: Let's start...\n");
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

12. Next, create the `Participant` class and specify that it implements the `Runnable` interface. This class represents each participant in the video conference:

```
public class Participant implements Runnable {
```

13. Declare a private `Videoconference` attribute named `conference`:

```
private Videoconference conference;
```

14. Declare a private `String` attribute named `name`:

```
private String name;
```

15. Implement the constructor of the class that initializes both the preceding attributes:

```
public Participant(Videoconference conference, String name) {  
    this.conference=conference;  
    this.name=name;  
}
```

16. Implement the `run()` method of the participants:

```
@Override  
public void run() {
```

17. First, put the thread to sleep for a random period of time:

```
    long duration=(long) (Math.random()*10);  
    try {  
        TimeUnit.SECONDS.sleep(duration);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }
```

18. Then, use the `arrive()` method of the `Videoconference` object to indicate the arrival of this participant:

```
        conference.arrive(name);
```

19. Finally, implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {  
    public static void main(String[] args) {
```

20. Next, create a `Videoconference` object named `conference` that waits for 10 participants:

```
        Videoconference conference=new Videoconference(10);
```

21. Create Thread to run this `Videoconference` object and start it:

```
        Thread threadConference=new Thread(conference);  
        threadConference.start();
```

22. Create 10 `Participant` objects, a `Thread` object to run each of them, and start all the threads:

```
for (int i=0; i<10; i++){
    Participant p=new Participant(conference, "Participant "+i);
    Thread t=new Thread(p);
    t.start();
}
```

How it works...

The `CountDownLatch` class has three basic elements:

- The initialization value that determines how many events the `CountDownLatch` object waits for
- The `await()` method, called by the threads that wait for the finalization of all the events
- The `countDown()` method, called by the events when they finish their execution

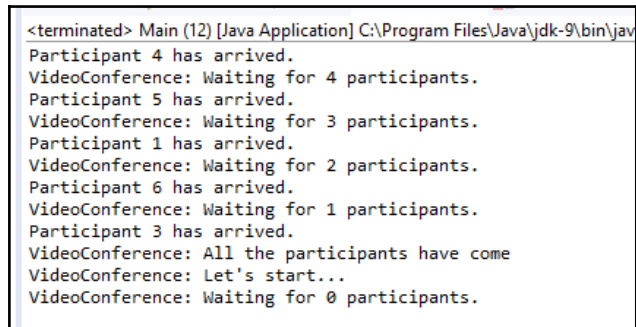
When you create a `CountDownLatch` object, it uses the constructor's parameter to initialize an internal counter. Every time a thread calls the `countDown()` method, the `CountDownLatch` object decrements the internal counter in one unit. When the internal counter reaches 0, the `CountDownLatch` object wakes up all the threads that were waiting in the `await()` method.

There's no way to re-initialize the internal counter of the `CountDownLatch` object or modify its value. Once the counter is initialized, the only method you can use to modify its value is the `countDown()` method explained earlier. When the counter reaches 0, all the calls to the `await()` method are returned immediately and all subsequent calls to the `countDown()` method have no effect.

However, there are some differences with respect to other synchronization methods, which are as follows:

- The `CountDownLatch` mechanism is not used to protect a shared resource or a critical section. It is used to synchronize one or more threads with the execution of various tasks.
- It only admits one use. As explained earlier, once the counter of `CountDownLatch` arrives at 0, all the calls to its methods have no effect. You have to create a new object if you want to do the same synchronization again.

The following screenshot shows the output of an execution of the example:



```
<terminated> Main (12) [Java Application] C:\Program Files\Java\jdk-9\bin\jav
Participant 4 has arrived.
VideoConference: Waiting for 4 participants.
Participant 5 has arrived.
VideoConference: Waiting for 3 participants.
Participant 1 has arrived.
VideoConference: Waiting for 2 participants.
Participant 6 has arrived.
VideoConference: Waiting for 1 participants.
Participant 3 has arrived.
VideoConference: All the participants have come
VideoConference: Let's start...
VideoConference: Waiting for 0 participants.
```

You can see how participants arrive, and once the internal counter arrives at 0, the `CountDownLatch` object wakes up the `Videoconference` object that writes the messages indicating that the video conference should start.

There's more...

The `CountDownLatch` class has another version of the `await()` method, which is as follows:

- `await(long time, TimeUnit unit)`: In this method, the thread will continue to sleep until it's interrupted, that is, either the internal counter of `CountDownLatch` arrives at 0 or the specified time passes. The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`.

Synchronizing tasks in a common point

The Java concurrency API provides a synchronizing utility that allows the synchronization of two or more threads at a determined point. It's the `CyclicBarrier` class. This class is similar to the `CountDownLatch` class explained in the *Waiting for multiple concurrent events* recipe in this chapter, but it presents some differences that make it a more powerful class.

The `CyclicBarrier` class is initialized with an integer number, which is the number of threads that will be synchronized at a determined point. When one of these threads arrives at the determined point, it calls the `await()` method to wait for the other threads. When the thread calls this method, the `CyclicBarrier` class blocks the thread that is sleeping until the other threads arrive. When the last thread calls the `await()` method of the `CyclicBarrier` object, it wakes up all the threads that were waiting and continues with its job.

One interesting advantage of the `CyclicBarrier` class is that you can pass an additional `Runnable` object as an initialization parameter, and the `CyclicBarrier` class executes this object as a thread when all the threads arrive at the common point. This characteristic makes this class adequate for parallelization of tasks using the divide and conquer programming technique.

In this recipe, you will learn how to use the `CyclicBarrier` class to synchronize a set of threads at a determined point. You will also use a `Runnable` object that will be executed after all the threads arrive at this point. In the example, you will look for a number in a matrix of numbers. The matrix will be divided into subsets (using the divide and conquer technique), so each thread will look for the number in one subset. Once all the threads have finished their respective jobs, a final task will unify their results.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Start the example by implementing two auxiliary classes. First, create a class named `MatrixMock`. This class will generate a random matrix of numbers between one and 10, where the threads will look for a number:

```
public class MatrixMock {
```

2. Declare a private `int` matrix named `data`:

```
private final int data[][];
```

3. Implement the constructor of the class. This constructor will receive the number of rows of the matrix, the length of each row, and the number we are going to look for as parameters. All the three parameters are of the type `int`:

```
public MatrixMock(int size, int length, int number){
```

4. Initialize the variables and objects used in the constructor:

```
int counter=0;
data=new int[size][length];
Random random=new Random();
```

5. Fill the matrix with random numbers. Each time you generate a number, compare it with the number you are going to look for. If they are equal, increment the counter:

```
for (int i=0; i<size; i++) {
    for (int j=0; j<length; j++){
        data[i][j]=random.nextInt(10);
        if (data[i][j]==number){
            counter++;
        }
    }
}
```

6. Finally, print a message in the console, which shows the number of occurrences of the number you are going to look for in the generated matrix. This message will be used to check that the threads get the correct result:

```
System.out.printf("Mock: There are %d occurrences of number in
generated data.\n", counter, number);
```

7. Implement the `getRow()` method. This method receives an `int` parameter with the number of a rows in the matrix; it returns the row if it exists and returns `null` if it doesn't:

```
public int[] getRow(int row){
    if ((row>=0)&&(row<data.length)){
        return data[row];
    }
    return null;
}
```


8. Now implement a class named `Results`. This class will store, in an array, the number of occurrences of the searched number in each row of the matrix:

```
public class Results {
```

9. Declare a private `int` array named `data`:

```
private final int data[];
```

10. Implement the constructor of the class. This constructor receives an integer parameter with the number of elements of the array:

```
public Results(int size){  
    data=new int[size];  
}
```

11. Implement the `setData()` method. This method receives a position in the array and a value as a parameter, and it establishes the value of that position in the array:

```
public void setData(int position, int value){  
    data[position]=value;  
}
```

12. Implement the `getData()` method. This method returns the array with the array of the results:

```
public int[] getData(){  
    return data;  
}
```

13. Now that you have the auxiliary classes, it's time to implement threads. First, implement the `Searcher` class. This class will look for a number in the determined rows of the matrix of random numbers. Create a class named `Searcher` and specify that it implements the `Runnable` interface:

```
public class Searcher implements Runnable {
```

14. Declare two private `int` attributes, namely `firstRow` and `lastRow`. These two attributes will determine the subset of rows where this object will look for the number:

```
private final int firstRow;  
private final int lastRow;
```

15. Declare a private `MatrixMock` attribute named `mock`:

```
private final MatrixMock mock;
```

16. Declare a private `Results` attribute named `results`:

```
private final Results results;
```

17. Declare a private `int` attribute named `number`, which will store the number we are going to look for:

```
private final int number;
```

18. Declare a `CyclicBarrier` object named `barrier`:

```
private final CyclicBarrier barrier;
```

19. Implement the constructor of the class that initializes all the attributes declared previously:

```
public Searcher(int firstRow, int lastRow, MatrixMock mock,
                Results results, int number, CyclicBarrier barrier){
    this.firstRow=firstRow;
    this.lastRow=lastRow;
    this.mock=mock;
    this.results=results;
    this.number=number;
    this.barrier=barrier;
}
```

20. Implement the `run()` method that will search for the number. It uses an internal variable called `counter` that will store the number of occurrences of the number in each row:

```
@Override
public void run() {
    int counter;
```

21. Print a message in the console with the rows assigned to this task:

```
System.out.printf("%s: Processing lines from %d to %d.\n",
                  Thread.currentThread().getName(),
                  firstRow, lastRow);
```

22. Process all the rows assigned to this thread. For each row, count the number of occurrences of the number you are searching for and store this number in the corresponding position of the `Results` object:

```
for (int i=firstRow; i<lastRow; i++){
    int row[]=mock.getRow(i);
    counter=0;
    for (int j=0; j<row.length; j++){
        if (row[j]==number){
            counter++;
        }
    }
    results.setData(i, counter);
}
```

23. Print a message in the console to indicate that this object has finished searching:

```
System.out.printf("%s: Lines processed.\n",
    Thread.currentThread().getName());
```

24. Call the `await()` method of the `CyclicBarrier` object and add the necessary code to process the `InterruptedException` and `BrokenBarrierException` exceptions that this method can throw:

```
try {
    barrier.await();
} catch (InterruptedException e) {
    e.printStackTrace();
} catch (BrokenBarrierException e) {
    e.printStackTrace();
}
```

25. Now implement the class that calculates the total number of occurrences of the number in the matrix. This class uses the `Results` object that stores the number of appearances of the number in each row of the matrix to make the calculation. Create a class named `Grouper` and specify that it implements the `Runnable` interface:

```
public class Grouper implements Runnable {
```

26. Declare a private `Results` attribute named `results`:

```
private final Results results;
```

27. Implement the constructor of the class that initializes the `Results` attribute:

```
public Grouper(Results results){
    this.results=results;
}
```

28. Implement the `run()` method that will calculate the total number of occurrences of the number in the array of results:

```
@Override
public void run() {
```

29. Declare an `int` variable and write a message to the console to indicate the start of the process:

```
int finalResult=0;
System.out.printf("Grouper: Processing results...\n");
```

30. Get the number of occurrences of the number in each row using the `getData()` method of the `results` object. Then, process all the elements of the array and add their value to the `finalResult` variable:

```
int data[]=results.getData();
for (int number:data){
    finalResult+=number;
}
```

31. Print the result in the console:

```
System.out.printf("Grouper: Total result: %d.\n", finalResult);
```

32. Finally, implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
    public static void main(String[] args) {
```

33. Declare and initialize five constants to store the parameters of the application:

```
final int ROWS=10000;
final int NUMBERS=1000;
final int SEARCH=5;
final int PARTICIPANTS=5;
final int LINES_PARTICIPANT=2000;
```

34. Create a `MatrixMock` object named `mock`. It will have 10,000 rows of 1,000 elements. Now, you are going to search for the number five:

```
MatrixMock mock=new MatrixMock(ROWS, NUMBERS, SEARCH);
```

35. Create a `Results` object named `results`. It will have 10,000 elements:

```
Results results=new Results(ROWS);
```

36. Create a `Grouper` object named `grouper`:

```
Grouper grouper=new Grouper(results);
```

37. Create a `CyclicBarrier` object called `barrier`. This object will wait for five threads. When these five threads finish, it will execute the `Grouper` object created previously:

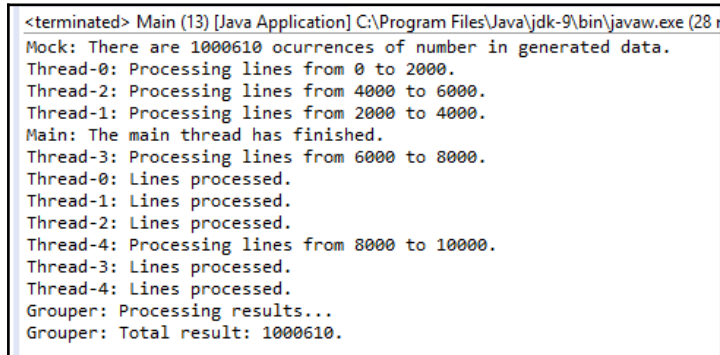
```
CyclicBarrier barrier=new CyclicBarrier(PARTICIPANTS,grouper);
```

38. Create five `Searcher` objects, five threads to execute them, and start the five threads:

```
Searcher searchers[]=new Searcher[PARTICIPANTS];
for (int i=0; i<PARTICIPANTS; i++){
    searchers[i]=new Searcher(i*LINES_PARTICIPANT,
                              (i*LINES_PARTICIPANT)+LINES_PARTICIPANT,
                              mock, results, 5,barrier);
    Thread thread=new Thread(searchers[i]);
    thread.start();
}
System.out.printf("Main: The main thread has finished.\n");
```

How it works...

The following screenshot shows the result of an execution of this example:



```
<terminated> Main (13) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe (28...
Mock: There are 1000610 occurrences of number in generated data.
Thread-0: Processing lines from 0 to 2000.
Thread-2: Processing lines from 4000 to 6000.
Thread-1: Processing lines from 2000 to 4000.
Main: The main thread has finished.
Thread-3: Processing lines from 6000 to 8000.
Thread-0: Lines processed.
Thread-1: Lines processed.
Thread-2: Lines processed.
Thread-4: Processing lines from 8000 to 10000.
Thread-3: Lines processed.
Thread-4: Lines processed.
Grouper: Processing results...
Grouper: Total result: 1000610.
```

The problem resolved in the example is simple. We have a big matrix of random integers, and you want to know the total number of occurrences of a number in this matrix. To get better performance, we used the divide and conquer technique. We divided the matrix into five subsets and used a thread to look for the number in each subset. These threads are objects of the `Searcher` class.

We used a `CyclicBarrier` object to synchronize the completion of the five threads and execute the `Grouper` task to process partial results and calculate the final one.

As mentioned earlier, the `CyclicBarrier` class has an internal counter to control how many threads need to arrive at the synchronization point. Each time a thread arrives at the synchronization point, it calls the `await()` method to notify the `CyclicBarrier` object that a thread has arrived at its synchronization point. `CyclicBarrier` puts the thread to sleep until all the threads reach the synchronization point.

When all the threads have arrived, the `CyclicBarrier` object wakes up all the threads that were waiting in the `await()` method. Optionally, it creates a new thread that executes a `Runnable` object passed as the parameter in the construction of `CyclicBarrier` (in our case, a `Grouper` object) to do additional tasks.

There's more...

The `CyclicBarrier` class has another version of the `await()` method:

- `await(long time, TimeUnit unit)`: In this method, the thread will continue to sleep until it's interrupted, that is, either the internal counter of `CyclicBarrier` arrives at 0 or the specified time passes. The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`.

This class also provides the `getNumberWaiting()` method that returns the number of threads that are blocked in the `await()` method and the `getParties()` method that returns the number of tasks that are going to be synchronized with `CyclicBarrier`.

Resetting a `CyclicBarrier` object

The `CyclicBarrier` class has some points in common with the `CountDownLatch` class, but they also have some differences. One of the most important differences is that a `CyclicBarrier` object can be reset to its initial state, assigning to its internal counter the value with which it was initialized.

This reset operation can be done using the `reset()` method of the `CyclicBarrier` class. When this occurs, all the threads that were waiting in the `await()` method receive a `BrokenBarrierException` exception. This exception was processed in the example presented in this recipe by printing the stack trace; however, in a more complex application, it could perform some other operation, such as restarting the execution or recovering the operation at the point it was interrupted.

Broken `CyclicBarrier` objects

A `CyclicBarrier` object can be in a special state denoted by the broken state. When there are various threads waiting in the `await()` method and one of them is interrupted, the one that is interrupted receives an `InterruptedException` exception, but other threads receive a `BrokenBarrierException` exception; `CyclicBarrier` is placed in the broken state.

The `CyclicBarrier` class provides the `isBroken()` method. It returns `true` if the object is in the broken state; otherwise, it returns `false`.

See also

- The *Waiting for multiple concurrent events* recipe in this chapter

Running concurrent-phased tasks

One of the most complex and powerful functionalities offered by the Java concurrency API is the ability to execute concurrent-phased tasks using the `Phaser` class. This mechanism is useful when we have some concurrent tasks divided into steps. The `Phaser` class provides us with a mechanism to synchronize threads at the end of each step, so no thread will start with the second step until all the threads have finished the first one.

As with other synchronization utilities, we have to initialize the `Phaser` class with the number of tasks that participate in the synchronization operation, but we can dynamically modify this number by either increasing or decreasing it.

In this recipe, you will learn how to use the `Phaser` class to synchronize three concurrent tasks. The three tasks look for files with the extension `.log` modified in the last 24 hours in three different folders and their subfolders. This task is divided into three steps:

1. Get a list of the files with the extension `.log` in the assigned folder and its subfolders.
2. Filter the list created in the first step by deleting the files modified more than 24 hours ago.
3. Print the results in the console.

At the end of step 1 and step 2, we check whether the list has any elements or not. If it doesn't, the thread ends its execution and is eliminated from the `Phaser` class.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `FileSearch` and specify that it implements the `Runnable` interface. This class implements the operation of searching for files with a determined extension modified in the last 24 hours in a folder and its subfolders:

```
public class FileSearch implements Runnable {
```

2. Declare a private `String` attribute to store the folder in which the search operation will begin:

```
private final String initPath;
```

3. Declare another private `String` attribute to store the extension of the files we are going to look for:

```
private final String fileExtension
```

4. Declare a private `List` attribute to store the full path of the files we will find with the desired characteristics:

```
private List<String> results;
```

5. Finally, declare a private `Phaser` attribute to control the synchronization of the different phases of the task:

```
private Phaser phaser;
```

6. Next, implement the constructor of the class that will initialize the attributes of the class. It receives the full path of the initial folder as parameters, the extension of the files, and `phaser`:

```
public FileSearch(String initPath, String fileExtension,
                  Phaser phaser) {
    this.initPath = initPath;
    this.fileExtension = fileExtension;
    this.phaser=phaser;
    results=new ArrayList<>();
}
```

7. Now, implement some auxiliary methods that will be used by the `run()` method. The first one is the `directoryProcess()` method. It receives a `File` object as a parameter and it processes all its files and subfolders. For each folder, the method will make a recursive call while passing the folder as a parameter. For each file, the method will call the `fileProcess()` method:

```
private void directoryProcess(File file) {

    File list[] = file.listFiles();
    if (list != null) {
        for (int i = 0; i < list.length; i++) {
            if (list[i].isDirectory()) {
                directoryProcess(list[i]);
            } else {
                fileProcess(list[i]);
            }
        }
    }
}
```

8. Then, implement the `fileProcess()` method. It receives a `File` object as a parameter and checks whether its extension is equal to the one we are looking for. If they are equal, this method adds the absolute path of the file to the list of results:

```
private void fileProcess(File file) {
    if (file.getName().endsWith(fileExtension)) {
        results.add(file.getAbsolutePath());
    }
}
```

9. Now implement the `filterResults()` method. It doesn't receive any parameter and filters the list of files obtained in the first phase; it deletes files that were modified more than 24 hours ago. First, create a new empty list and get the actual date:

```
private void filterResults() {
    List<String> newResults=new ArrayList<>();
    long actualDate=new Date().getTime();
```

10. Then, go through all the elements of the results list. For each path in the list of results, create a `File` object for the file and get its last modified date:

```
for (int i=0; i<results.size(); i++){
    File file=new File(results.get(i));
```

```
long fileDate=file.lastModified();
```

11. Then, compare this date with the actual date, and if the difference is less than 1 day, add the full path of the file to the new list of results:

```
if (actualDate-fileDate< TimeUnit.MILLISECONDS
    .convert(1,TimeUnit.DAYS)){
    newResults.add(results.get(i));
}
}
```

12. Finally, change the old results list to the new ones:

```
results=newResults;
}
```

13. Next, implement the `checkResults()` method. This method will be called at the end of the first and second phase, and it will check whether the results list is empty or not. This method doesn't have any parameters:

```
private boolean checkResults() {
```

14. First, check the size of the results list. If it's 0, the object writes a message to the console indicating this. After this, it calls the `arriveAndDeregister()` method of the `Phaser` object to notify that this thread has finished the actual phase and it leaves the phased operation:

```
if (results.isEmpty()) {
    System.out.printf("%s: Phase %d: 0 results.\n",
        Thread.currentThread().getName(),
        phaser.getPhase());
    System.out.printf("%s: Phase %d: End.\n",
        Thread.currentThread().getName(),
        phaser.getPhase());
    phaser.arriveAndDeregister();
    return false;
```

15. If the results list has elements, the object writes a message to the console indicating this. Then, it calls the `arriveAndAwaitAdvance()` method of the `Phaser` object to notify that this thread has finished the actual phase and it wants to be blocked until all the participant threads in the phased operation finish the actual phase:

```
    } else {
        System.out.printf("%s: Phase %d: %d results.\n",
            Thread.currentThread().getName(),
```

```
        phaser.getPhase(), results.size());
    phaser.arriveAndAwaitAdvance();
    return true;
}
}
```

16. The last auxiliary method is the `showInfo()` method that prints the elements of the results list to the console:

```
private void showInfo() {
    for (int i=0; i<results.size(); i++){
        File file=new File(results.get(i));
        System.out.printf("%s: %s\n",
                           Thread.currentThread().getName(),
                           file.getAbsolutePath());
    }
    phaser.arriveAndAwaitAdvance();
}
```

17. It's time to implement the `run()` method that executes the operation using the auxiliary methods described earlier. We'll also implement the `Phaser` object to control the change between phases. First, call the `arriveAndAwaitAdvance()` method of the `Phaser` object. The search won't begin until all the threads have been created:

```
@Override
public void run() {
    phaser.arriveAndAwaitAdvance();
}
```

18. Then, write a message to the console indicating the start of the search task:

```
System.out.printf("%s: Starting.\n",
                  Thread.currentThread().getName());
```

19. Check that the `initPath` attribute stores the name of a folder and use the `directoryProcess()` method to find the files with the specified extension in that folder and all its subfolders:

```
File file = new File(initPath);
if (file.isDirectory()) {
    directoryProcess(file);
}
```

20. Check whether there are any results using the `checkResults()` method. If there are no results, finish the execution of the thread with the `return` keyword:

```
if (!checkResults()) {
    return;
}
```

21. Filter the list of results using the `filterResults()` method:

```
filterResults();
```

22. Check whether there are any results using the `checkResults()` method once again. If there are no results, finish the execution of the thread with the `return` keyword:

```
if (!checkResults()) {
    return;
}
```

23. Print the final list of results to the console with the `showInfo()` method, deregister the thread, and print a message indicating the finalization of the thread:

```
showInfo();
phaser.arriveAndDeregister();
System.out.printf("%s: Work completed.\n",
                  Thread.currentThread().getName());
```

24. Now, implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
    public static void main(String[] args) {
```

25. Create a `Phaser` object with three participants:

```
Phaser phaser=new Phaser(3);
```

26. Create three `FileSearch` objects with a different initial folder for each one. Look for the files with the `.log` extension:

```
FileSearch system=new FileSearch("C:\\Windows", "log", phaser);
FileSearch apps= new FileSearch("C:\\Program Files",
                                "log",phaser);
FileSearch documents= new FileSearch("C:\\Documents And Settings",
                                     "log",phaser);
```

27. Create and start a thread to execute the first `FileSearch` object:

```
Thread systemThread=new Thread(system,"System");
systemThread.start();
```

28. Create and start a thread to execute the second `FileSearch` object:

```
Thread appsThread=new Thread(apps,"Apps");
appsThread.start();
```

29. Create and start a thread to execute the third `FileSearch` object:

```
Thread documentsThread=new Thread(documents, "Documents");
documentsThread.start();
```

30. Wait for the finalization of the three threads:

```
try {
    systemThread.join();
    appsThread.join();
    documentsThread.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

31. Write the value of the finalized flag of the `Phaser` object using the `isFinalized()` method:

```
System.out.println("Terminated: "+ phaser.isTerminated());
```

How it works...

The program starts creating a `Phaser` object that will control the synchronization of the threads at the end of each phase. The constructor of `Phaser` receives the number of participants as a parameter. In our case, `Phaser` has three participants. This number indicates `Phaser` the number of threads that need to execute the `arriveAndAwaitAdvance()` method before `Phaser` could change the phase and wake up the threads that have been sleeping.

Once `Phaser` has been created, we launch three threads that are executed using three different `FileSearch` objects.



In this example, we use paths of the Windows operating system. If you work with another operating system, modify the paths to adapt them to existing paths in your environment, such as `/var/log`, or similar.

The first instruction in the `run()` method of this `FileSearch` object is a call to the `arriveAndAwaitAdvance()` method of the `Phaser` object. As mentioned earlier, `Phaser` knows the number of threads that we want to synchronize. When a thread calls this method, `Phaser` decreases the number of threads that have to finalize the actual phase and puts this thread to sleep until all the remaining threads finish this phase. Calling this method at the beginning of the `run()` method ensures that none of the `FileSearch` threads begin their job until all the threads are created.

At the end of phase one and phase two, we check whether the phase has generated results and the list with the results has elements or the phase hasn't generated results and the list is empty. In the first case, the `checkResults()` method calls `arriveAndAwaitAdvance()` as explained earlier. In the second case, if the list is empty, there's no point in the thread continuing with its execution, so it ends its execution. But you have to notify the `Phaser` object that there will be one less participant. For this, we used `arriveAndDeregister()`. This notifies `phaser` that the thread has finished the actual phase, but it won't participate in future phases, therefore, `phaser` won't have to wait for it to continue.

At the end of the phase three implemented in the `showInfo()` method, there is a call to the `arriveAndAwaitAdvance()` method of `phaser`. With this call, we guarantee that all the threads finish at the same time. When this method ends its execution, there is a call to the `arriveAndDeregister()` method of `phaser`. With this call, we deregister the threads of `phaser`, as explained before, so when all the threads finish, `phaser` will have zero participants.

Finally, the `main()` method waits for the completion of the three threads and calls the `isTerminated()` method of `phaser`. When `phaser` has zero participants, it enters the so-called termination state, and this method returns `true`. As we deregister all the threads of `phaser`, it will be in the termination state, and this call will print `true` to the console.

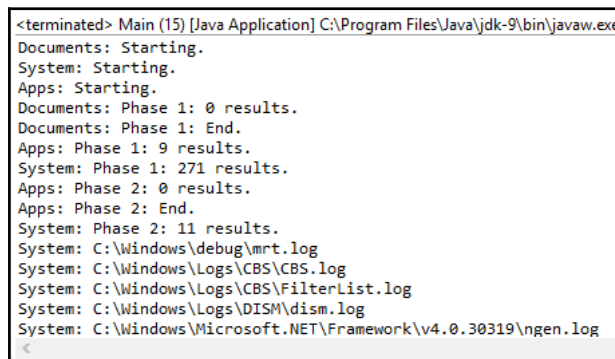
A `Phaser` object can be in two states:

- **Active:** `Phaser` enters this state when it accepts the registration of new participants and its synchronization at the end of each phase. In this state, `Phaser` works as it has been explained in this recipe. This state is not mentioned in the Java concurrency API.

- **Termination:** By default, `Phaser` enters this state when all the participants in `Phaser` have been deregistered, which means it has zero participants. Further, `Phaser` is in the termination state when the method `onAdvance()` returns `true`. If you override this method, you can change the default behavior. When `Phaser` is in this state, the synchronization method `arriveAndAwaitAdvance()` returns immediately without doing any synchronization operation.

A notable feature of the `Phaser` class is that you haven't had to control any exception from the methods related to `phaser`. Unlike other synchronization utilities, threads that are sleeping in `phaser` don't respond to interruption events and don't throw an `InterruptedException` exception. There is only one exception, which is explained in the next section.

The following screenshot shows the results of one execution of the example:



```
<terminated> Main (15) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe
Documents: Starting.
System: Starting.
Apps: Starting.
Documents: Phase 1: 0 results.
Documents: Phase 1: End.
Apps: Phase 1: 9 results.
System: Phase 1: 271 results.
Apps: Phase 2: 0 results.
Apps: Phase 2: End.
System: Phase 2: 11 results.
System: C:\Windows\debug\mrt.log
System: C:\Windows\Logs\CBS\CBS.log
System: C:\Windows\Logs\CBS\FilterList.log
System: C:\Windows\Logs\DISM\dism.log
System: C:\Windows\Microsoft.NET\Framework\v4.0.30319\ngen.log
<
```

It shows the first two phases of the execution. You can see how the `Apps` thread finishes its execution in phase two because its results list is empty. When you execute the example, you will see how some threads finish a phase before the rest and how they wait until all have finished one phase before continuing with the rest.

There's more...

The `Phaser` class provides other methods related to the change of phase. These methods are as follows:

- `arrive()`: This method notifies the `Phaser` class that one participant has finished the actual phase but it should not wait for the rest of the participants to continue with their execution. Be careful with the utilization of this method because it doesn't synchronize with other threads.

- `awaitAdvance(int phase)`: This method puts the current thread to sleep until all the participants of the `phaser` parameter have finished the current phase, that is, if the number we pass as the parameter is equal to the actual phase of `phaser`. If the parameter and the actual phase of `phaser` aren't equal, the method ends its execution.
- `awaitAdvanceInterruptibly(int phaser)`: This method is equal to the method explained earlier, but it throws an `InterruptedException` exception if the thread that is sleeping in this method is interrupted.

Registering participants in Phaser

When you create a `Phaser` object, you indicate how many participants will have that `phaser`. But the `Phaser` class has two methods to increment the number of participants of `phaser`. These methods are as follows:

- `register()`: This method adds a new participant to `Phaser`. This new participant will be considered unrarried to the actual phase.
- `bulkRegister(int Parties)`: This method adds the specified number of participants to `phaser`. These new participants will be considered unrarried to the actual phase.

The only method provided by the `Phaser` class to decrement the number of participants is the `arriveAndDeregister()` method that notifies `phaser` that the thread has finished the actual phase and it doesn't want to continue with the phased operation.

Forcing the termination of Phaser

When `phaser` has zero participants, it enters a state denoted by termination. The `Phaser` class provides `forceTermination()` to change the status of `phaser` and makes it enter the termination state independently of the number of participants registered in `phaser`. This mechanism may be useful when one of the participants has an error situation, where the best thing to do would be to terminate `phaser`.

When `phaser` is in the termination state, the `awaitAdvance()` and `arriveAndAwaitAdvance()` methods immediately return a negative number, instead of a positive one that is returned normally. If you know that your `phaser` could be terminated, you should verify the return value of those methods (`awaitAdvance()` and `arriveAndAwaitAdvance()`) to know whether `phaser` has been terminated.

See also

- The *Monitoring a Phaser class* recipe in Chapter 9, *Testing Concurrent Applications*

Controlling phase change in concurrent-phased tasks

The `Phaser` class provides a method that is executed each time `phaser` changes the phase. It's the `onAdvance()` method. It receives two parameters: the number of the current phases and the number of registered participants. It returns a Boolean value `false` if `Phaser` continues its execution or the value `true` if `Phaser` has finished and has to enter the termination state.

The default implementation of this method returns `true` if the number of registered participants is zero, and `false` otherwise. But you can modify this behavior if you extend the `Phaser` class and override this method. Normally, you will be interested in doing this when you have to execute some actions when you advance from one phase to the next.

In this recipe, you will learn how to control phase change in a phaser that is implementing your own version of the `Phaser` class that overrides the `onAdvance()` method to execute some actions in every phase change. You are going to implement a simulation of an exam, where there will be some students who have to do three exercises. All the students have to finish one exercise before they proceed with the next one.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `MyPhaser` and specify that it extends from the `Phaser` class:

```
public class MyPhaser extends Phaser {
```

2. Override the `onAdvance()` method. According to the value of the `phase` attribute, we call it a different auxiliary method. If the `phase` attribute is equal to zero, you have to call the `studentsArrived()` method. If the `phase` is equal to one, you have to call the `finishFirstExercise()` method. If the `phase` is equal to two, you have to call the `finishSecondExercise()` method. Finally, if the `phase` is equal to three, you have to call the `finishExam()` method. Otherwise, return the true value to indicate that phaser has terminated:

```
@Override
protected boolean onAdvance(int phase, int registeredParties) {
    switch (phase) {
        case 0:
            return studentsArrived();
        case 1:
            return finishFirstExercise();
        case 2:
            return finishSecondExercise();
        case 3:
            return finishExam();
        default:
            return true;
    }
}
```

3. Implement the auxiliary method `studentsArrived()`. It writes two log messages to the console and returns false to indicate that phaser is continuing with its execution:

```
private boolean studentsArrived() {
    System.out.printf("Phaser: The exam are going to start.
                      The students are ready.\n");
    System.out.printf("Phaser: We have %d students.\n",
                      getRegisteredParties());
    return false;
}
```

4. Implement the auxiliary method `finishFirstExercise()`. It writes two messages to the console and returns `false` to indicate that `phaser` is continuing with its execution:

```
private boolean finishFirstExercise() {
    System.out.printf("Phaser: All the students have finished the
                      first exercise.\n");
    System.out.printf("Phaser: It's time for the second one.\n");
    return false;
}
```

5. Implement the auxiliary method `finishSecondExercise()`. It writes two messages to the console and returns `false` to indicate that `phaser` is continuing with its execution:

```
private boolean finishSecondExercise() {
    System.out.printf("Phaser: All the students have finished the
                      second exercise.\n");
    System.out.printf("Phaser: It's time for the third one.\n");
    return false;
}
```

6. Implement the auxiliary method `finishExam()`. It writes two messages to the console and returns `true` to indicate that `phaser` has finished its work:

```
private boolean finishExam() {
    System.out.printf("Phaser: All the students have finished
                      the exam.\n");
    System.out.printf("Phaser: Thank you for your time.\n");
    return true;
}
```

7. Create a class named `Student` and specify that it implements the `Runnable` interface. This class will simulate the students of an exam:

```
public class Student implements Runnable {
```

8. Declare a `Phaser` object named `phaser`:

```
private Phaser phaser;
```

9. Implement the constructor of the class that initializes the `Phaser` object:

```
public Student(Phaser phaser) {
    this.phaser=phaser;
}
```

10. Implement the `run()` method that will simulate the realization of the exam:

```
@Override
public void run() {
```

11. First, the method writes a message in the console to indicate that a student has arrived at the exam hall and calls the `arriveAndAwaitAdvance()` method of `phaser` to wait for the rest of the threads:

```
System.out.printf("%s: Has arrived to do the exam. %s\n",
                  Thread.currentThread().getName(), new Date());
phaser.arriveAndAwaitAdvance();
```

12. Then, write a message to the console and call the private `doExercise1()` method that simulates the realization of the first exercise of the exam. Post this, write another message to the console and the `arriveAndAwaitAdvance()` method of `phaser` to wait for the rest of the students to finish the first exercise:

```
System.out.printf("%s: Is going to do the first exercise.%s\n",
                  Thread.currentThread().getName(), new Date());
doExercise1();
System.out.printf("%s: Has done the first exercise.%s\n",
                  Thread.currentThread().getName(), new Date());
phaser.arriveAndAwaitAdvance();
```

13. Implement the same code for the second and third exercises:

```
System.out.printf("%s: Is going to do the second exercise.
                  %s\n", Thread.currentThread().getName(),
                  new Date());
doExercise2();
System.out.printf("%s: Has done the second exercise.%s\n",
                  Thread.currentThread().getName(), new Date());
phaser.arriveAndAwaitAdvance();
System.out.printf("%s: Is going to do the third exercise.%s\n",
                  Thread.currentThread().getName(), new Date());
doExercise3();
System.out.printf("%s: Has finished the exam.%s\n",
                  Thread.currentThread().getName(), new Date());
phaser.arriveAndAwaitAdvance();
```

14. Implement the auxiliary method `doExercise1()`. This method puts the current thread or the thread that executes the method to sleep for a random period of time:

```
private void doExercise1() {
    try {
        long duration=(long) (Math.random()*10);
        TimeUnit.SECONDS.sleep(duration);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

15. Implement the auxiliary method `doExercise2()`. This method puts the current thread or the thread that executes the method to sleep for a random period of time:

```
private void doExercise2() {
    try {
        long duration=(long) (Math.random()*10);
        TimeUnit.SECONDS.sleep(duration);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

16. Implement the auxiliary method `doExercise3()`. This method puts the thread to sleep for a random period of time:

```
private void doExercise3() {
    try {
        long duration=(long) (Math.random()*10);
        TimeUnit.SECONDS.sleep(duration);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

17. Implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
    public static void main(String[] args) {
```

18. Create a `MyPhaser` object:

```
MyPhaser phaser=new MyPhaser();
```

19. Create five `Student` objects and register them in the `phaser` attribute using the `register()` method:

```
Student students[]=new Student[5];
for (int i=0; i<students.length; i++){
    students[i]=new Student(phaser);
    phaser.register();
}
```

20. Create five threads to run students and start them:

```
Thread threads[]=new Thread[students.length];
for (int i=0; i<students.length; i++){
    threads[i]=new Thread(students[i], "Student "+i);
    threads[i].start();
}
```

21. Wait for the finalization of the five threads:

```
for (int i=0; i<threads.length; i++){
    try {
        threads[i].join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

22. Write a message to show that `phaser` is in the termination state, using the `isTerminated()` method:

```
System.out.printf("Main: The phaser has finished: %s.\n",
    phaser.isTerminated());
```

How it works...

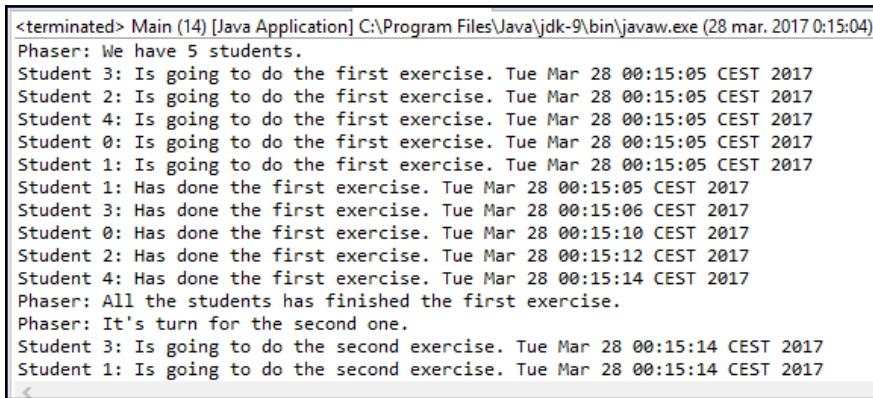
This exercise simulates the realization of an exam that has three exercises. All the students have to finish one exercise before they can start the next one. To implement this synchronization requirement, we use the `Phaser` class; however, in this case, you implemented your own `phaser`, extending the original class to override the `onAdvance()` method.

This method is called by `Phaser` before making a phase change and waking up all the threads that were sleeping in the `arriveAndAwaitAdvance()` method. The method is invoked by the last thread that finishes a phase as part of the code of the `arriveAndAwaitAdvance()` method. This method receives the number of the actual phase as parameters, where 0 is the number of the first phase and the number of registered participants. The most useful parameter is the actual phase. If you execute a different operation depending on the actual phase, you have to use an alternative structure (`if...else` or `switch`) to select the operation you want to execute. In the example, we used a `switch` structure to select a different method for each change of phase.

The `onAdvance()` method returns a Boolean value that indicates whether `phaser` has terminated or not. If `phaser` returns `false`, it indicates that it hasn't terminated; if this happens, the threads will continue with the execution of other phases. If `phaser` returns `true`, then `phaser` still wakes up the pending threads but moves `phaser` to the terminated state. With this, all future calls to any method of `phaser` will return immediately, and the `isTerminated()` method will return `true`.

In the `Main` class, when you created the `MyPhaser` object, you didn't specify the number of participants in the phaser. You made a call to the `register()` method for every `Student` object created to register a participant in `phaser`. This calling doesn't establish a relation between the `Student` object or the thread that executes it and `phaser`. Really, the number of participants in a phaser is only a number. There is no relationship between `phaser` and the participants.

The following screenshot shows the results of an execution of this example:



```
<terminated> Main (14) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe (28 mar. 2017 0:15:04)
Phaser: We have 5 students.
Student 3: Is going to do the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 2: Is going to do the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 4: Is going to do the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 0: Is going to do the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 1: Is going to do the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 1: Has done the first exercise. Tue Mar 28 00:15:05 CEST 2017
Student 3: Has done the first exercise. Tue Mar 28 00:15:06 CEST 2017
Student 0: Has done the first exercise. Tue Mar 28 00:15:10 CEST 2017
Student 2: Has done the first exercise. Tue Mar 28 00:15:12 CEST 2017
Student 4: Has done the first exercise. Tue Mar 28 00:15:14 CEST 2017
Phaser: All the students has finished the first exercise.
Phaser: It's turn for the second one.
Student 3: Is going to do the second exercise. Tue Mar 28 00:15:14 CEST 2017
Student 1: Is going to do the second exercise. Tue Mar 28 00:15:14 CEST 2017
<
```


You can see how the students finished the first exercise at different times. When all of them finish the first exercise, `phaser` calls the `onAdvance()` method that writes the log messages in the console, then all the students start the second exercise at the same time.

See also

- The *Running concurrent-phased tasks* recipe in this chapter
- The *Monitoring a Phaser class* recipe in Chapter 9, *Testing Concurrent Applications*

Exchanging data between concurrent tasks

The Java concurrency API provides a synchronization utility that allows interchange of data between two concurrent tasks. In more detail, the `Exchanger` class allows you to have a definition of a synchronization point between two threads. When the two threads arrive at this point, they interchange a data structure such that the data structure of the first thread goes to the second one and vice versa.

This class may be very useful in a situation similar to the producer-consumer problem. This is a classic concurrent problem where you have a common buffer of data, one or more producers of data, and one or more consumers of data. As the `Exchanger` class synchronizes only two threads, you can use it if you have a producer-consumer problem with one producer and one consumer.

In this recipe, you will learn how to use the `Exchanger` class to solve the producer-consumer problem with one producer and one consumer.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, begin by implementing the producer. Create a class named `Producer` and specify that it implements the `Runnable` interface:

```
public class Producer implements Runnable {
```

2. Declare a `List<String>` field named `buffer`. This will be the data structure that the producer will interchange with the consumer:

```
private List<String> buffer;
```

3. Declare an `Exchanger<List<String>>` field named `exchanger`. This will be the exchanger object that will be used to synchronize the producer and consumer:

```
private final Exchanger<List<String>> exchanger;
```

3. Implement the constructor of the class that will initialize the two attributes:

```
public Producer (List<String> buffer, Exchanger<List<String>>
                exchanger) {
    this.buffer=buffer;
    this.exchanger=exchanger;
}
```

4. Implement the `run()` method. Inside it, implement 10 cycles of interchange:

```
@Override
public void run() {
    for (int cycle = 1; cycle <= 10; cycle++){
        System.out.printf("Producer: Cycle %d\n",cycle);
```

5. In each cycle, add 10 strings to the buffer:

```
for (int j=0; j<10; j++){
    String message="Event "+((cycle-1)*10)+j);
    System.out.printf("Producer: %s\n",message);
    buffer.add(message);
}
```

6. Call the `exchange()` method to interchange data with the consumer. As this method can throw an `InterruptedException` exception, you have to add some code to process it.

```
try {
    buffer=exchanger.exchange(buffer);
} catch (InterruptedException e) {
    e.printStackTrace();
}
System.out.println("Producer: "+buffer.size());
}
```

7. Now, implement the consumer. Create a class named `Consumer` and specify that it implements the `Runnable` interface:

```
public class Consumer implements Runnable {
```

8. Declare a `List<String>` field named `buffer`. This will be the data structure that the producer will interchange with the consumer:

```
private List<String> buffer;
```

9. Declare an `Exchanger<List<String>>` field named `exchanger`. This will be the `exchanger` object that will be used to synchronize the producer and consumer:

```
private final Exchanger<List<String>> exchanger;
```

10. Implement the constructor of the class that will initialize the two attributes:

```
public Consumer(List<String> buffer, Exchanger<List<String>>
                exchanger) {
    this.buffer=buffer;
    this.exchanger=exchanger;
}
```

11. Implement the `run()` method. Inside it, implement 10 cycles of interchange:

```
@Override
public void run() {
    for (int cycle=1; cycle <= 10; cycle++){
        System.out.printf("Consumer: Cycle %d\n",cycle);
    }
}
```

12. In each cycle, begin with a call to the `exchange()` method to synchronize with the producer. The consumer needs data to consume. As this method can throw an `InterruptedException` exception, you have to add some code to process it:

```
try {
    buffer=exchanger.exchange(buffer);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

13. Write the 10 strings the producer sent in its buffer to the console and delete them from the buffer to leave it empty:

```
System.out.println("Consumer: "+buffer.size());
for (int j=0; j<10; j++){
    String message=buffer.get(0);
    System.out.println("Consumer: "+message);
    buffer.remove(0);
}
}
```

14. Now, implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
    public static void main(String[] args) {
```

15. Create two buffers that will be used by the producer and consumer:

```
List<String> buffer1=new ArrayList<>();
List<String> buffer2=new ArrayList<>();
```

16. Create the `Exchanger` object that will be used to synchronize the producer and consumer:

```
Exchanger<List<String>> exchanger=new Exchanger<>();
```

17. Create the `Producer` and `Consumer` objects:

```
Producer producer=new Producer(buffer1, exchanger);
Consumer consumer=new Consumer(buffer2, exchanger);
```

18. Create the threads to execute the producer and consumer and start the threads:

```
Thread threadProducer=new Thread(producer);
Thread threadConsumer=new Thread(consumer);

threadProducer.start();
threadConsumer.start();
```

How it works...

The consumer begins with an empty buffer and calls `Exchanger` to synchronize with the producer. It needs data to consume. The producer begins its execution with an empty buffer. It creates 10 strings, stores them in the buffer, and uses the `Exchanger` to synchronize with the consumer.

At this point, both the threads (producer and consumer) are in `Exchanger`, which changes the data structures. So when the consumer returns from the `exchange()` method, it will have a buffer with 10 strings. When the producer returns from the `exchange()` method, it will have an empty buffer to fill again. This operation will be repeated 10 times.

If you execute the example, you will see how producer and consumer do their jobs concurrently and how the two objects interchange their buffers in every step. As it occurs with other synchronization utilities, the first thread that calls the `exchange()` method is put to sleep until the other threads arrive.

There's more...

The `Exchanger` class has another version of the `exchange` method: `exchange(V data, long time, TimeUnit unit)`. where, `V` is the type used as a parameter in the declaration of `Phaser` (`List<String>` in our case). The thread will sleep until it's interrupted, another thread arrives, or the specified time passes. In this case, a `TimeoutException` is thrown. The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`.

Completing and linking tasks asynchronously

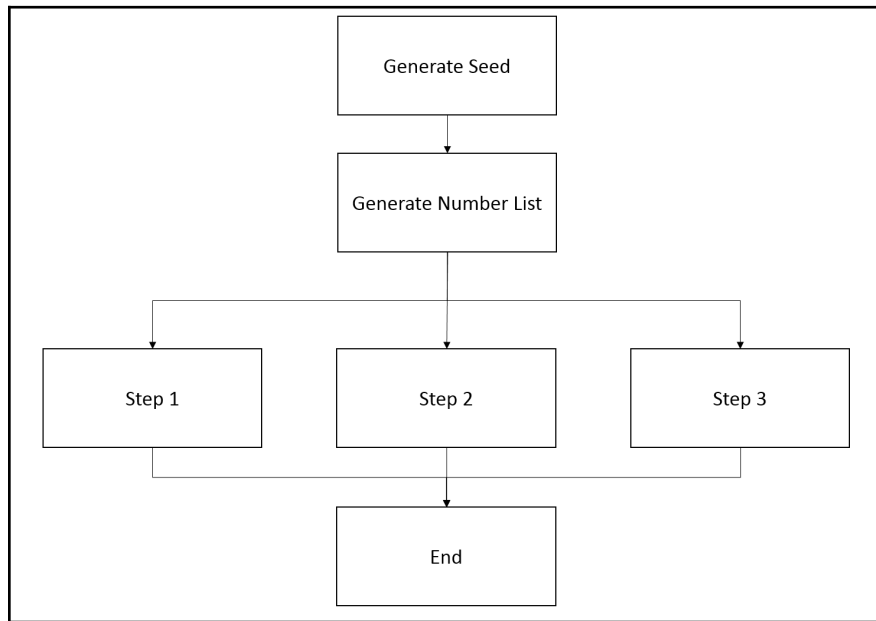
Java 8 Concurrency API includes a new synchronization mechanism with the `CompletableFuture` class. This class implements the `Future` object and the `CompletionStage` interface that gives it the following two characteristics:

- As the `Future` object, a `CompletableFuture` object will return a result sometime in future
- As the `CompletionStage` object, you can execute more asynchronous tasks after the completion of one or more `CompletableFuture` objects

You can work with a `CompletableFuture` class in different ways:

- You can create a `CompletableFuture` object explicitly and use it as a synchronization point between tasks. One task will establish the value returned by `CompletableFuture`, using the `complete()` method, and the other tasks will wait for this value, using the `get()` or `join()` methods.
- You can use a static method of the `CompletableFuture` class to execute `Runnable` or `Supplier` with the `runAsync()` and `supplyAsync()` methods. These methods will return a `CompletableFuture` object that will be completed when these tasks end their execution. In the second case, the value returned by `Supplier` will be the completion value of `CompletableFuture`.
- You can specify other tasks to be executed in an asynchronous way after the completion of one or more `CompletableFuture` objects. This task can implement the `Runnable`, `Function`, `Consumer` or `BiConsumer` interfaces.

These characteristics make the `CompletableFuture` class very flexible and powerful. In this chapter, you will learn how to use this class to organize different tasks. The main purpose of the example is that the tasks will be executed, as specified in the following diagram:



First, we're going to create a task that will generate a seed. Using this seed, the next task will generate a list of random numbers. Then, we will execute three parallel tasks:

1. Step 1 will calculate the nearest number to 1,000, in a list of random numbers.
2. Step 2 will calculate the biggest number in a list of random numbers.
3. Step 3 will calculate the average number between the largest and smallest numbers in a list of random numbers.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, we're going to implement the auxiliary tasks we will use in the example. Create a class named `SeedGenerator` that implements the `Runnable` interface. It will have a `CompletableFuture` object as an attribute, and it will be initialized in the constructor of the class:

```
public class SeedGenerator implements Runnable {  
  
    private CompletableFuture<Integer> resultCommunicator;  
  
    public SeedGenerator (CompletableFuture<Integer> completable) {  
        this.resultCommunicator=completable;  
    }  
}
```

2. Then, implement the `run()` method. It will sleep the current thread for 5 seconds (to simulate a long operation), calculate a random number between 1 and 10, and then use the `complete()` method of the `resultCommunicator` object to complete `CompletableFuture`:

```
@Override  
public void run() {  
  
    System.out.printf("SeedGenerator: Generating seed...\n");  
    // Wait 5 seconds  
    try {  
        TimeUnit.SECONDS.sleep(5);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    int seed=(int) Math rint(Math.random() * 10);  
  
    System.out.printf("SeedGenerator: Seed generated: %d\n",  
        seed);  
  
    resultCommunicator.complete(seed);  
}
```

3. Create a class named `NumberListGenerator` that implements the `Supplier` interface parameterized with the `List<Long>` data type. This means that the `get()` method provided by the `Supplier` interface will return a list of large numbers. This class will have an integer number as a private attribute, which will be initialized in the constructor of the class:


```
public class NumberListGenerator implements Supplier<List<Long>> {  
  
    private final int size;  
  
    public NumberListGenerator (int size) {  
        this.size=size;  
    }  
}
```

4. Then, implement the `get ()` method that will return a list with millions of numbers, as specified in the size parameter of larger random numbers:

```
@Override  
public List<Long> get() {  
    List<Long> ret = new ArrayList<>();  
    System.out.printf("%s : NumberListGenerator : Start\n",  
        Thread.currentThread().getName());  
  
    for (int i=0; i< size*1000000; i++) {  
        long number=Math.round(Math.random()*Long.MAX_VALUE);  
        ret.add(number);  
    }  
    System.out.printf("%s : NumberListGenerator : End\n",  
        Thread.currentThread().getName());  
  
    return ret;  
}
```

5. Finally, create a class named `NumberSelector` that implements the `Function` interface parameterized with the `List<Long>` and `Long` data types. This means that the `apply ()` method provided by the `Function` interface will receive a list of large numbers and will return a `Long` number:

```
public class NumberSelector implements Function<List<Long>, Long> {  
  
    @Override  
    public Long apply(List<Long> list) {  
  
        System.out.printf("%s: Step 3: Start\n",  
            Thread.currentThread().getName());  
        long max=list.stream().max(Long::compare).get();  
        long min=list.stream().min(Long::compare).get();  
        long result=(max+min)/2;  
        System.out.printf("%s: Step 3: Result - %d\n",  
            Thread.currentThread().getName(), result);  
        return result;  
    }  
}
```

6. Now it's time to implement the `Main` class and the `main()` method:

```
public class Main {
    public static void main(String[] args) {
```

7. First, create a `CompletableFuture` object and a `SeedGenerator` task and execute it as a `Thread`:

```
System.out.printf("Main: Start\n");
CompletableFuture<Integer> seedFuture = new CompletableFuture<>();
Thread seedThread = new Thread(new SeedGenerator(seedFuture));
seedThread.start();
```

8. Then, wait for the seed generated by the `SeedGenerator` task, using the `get()` method of the `CompletableFuture` object:

```
System.out.printf("Main: Getting the seed\n");
int seed = 0;
try {
    seed = seedFuture.get();
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
System.out.printf("Main: The seed is: %d\n", seed);
```

9. Now create another `CompletableFuture` object to control the execution of a `NumberListGenerator` task, but in this case, use the static method `supplyAsync()`:

```
System.out.printf("Main: Launching the list of numbers
                  generator\n");
NumberListGenerator task = new NumberListGenerator(seed);
CompletableFuture<List<Long>> startFuture = CompletableFuture
    .supplyAsync(task);
```

10. Then, configure the three parallelized tasks that will make calculations based on the list of numbers generated in the previous task. These three steps can't start their execution until the `NumberListGenerator` task has finished its execution, so we use the `CompletableFuture` object generated in the previous step and the `thenApplyAsync()` method to configure these tasks. The first two steps are implemented in a functional way, and the third one is an object of the `NumberSelector` class:

```
System.out.printf("Main: Launching step 1\n");
CompletableFuture<Long> step1Future = startFuture
    .thenApplyAsync(list -> {
        System.out.printf("%s: Step 1: Start\n",
            Thread.currentThread().getName());
        long selected = 0;
        long selectedDistance = Long.MAX_VALUE;
        long distance;
        for (Long number : list) {
            distance = Math.abs(number - 1000);
            if (distance < selectedDistance) {
                selected = number;
                selectedDistance = distance;
            }
        }
        System.out.printf("%s: Step 1: Result - %d\n",
            Thread.currentThread().getName(), selected);
        return selected;
    });

System.out.printf("Main: Launching step 2\n");
CompletableFuture<Long> step2Future = startFuture
    .thenApplyAsync(list -> list.stream().max(Long::compare).get());

CompletableFuture<Void> write2Future = step2Future
    .thenAccept(selected -> {
        System.out.printf("%s: Step 2: Result - %d\n",
            Thread.currentThread().getName(), selected);
    });

System.out.printf("Main: Launching step 3\n");
NumberSelector numberSelector = new NumberSelector();
CompletableFuture<Long> step3Future = startFuture
    .thenApplyAsync(numberSelector);
```

11. We wait for the finalization of the three parallel steps with the `allOf()` static method of the `CompletableFuture` class:

```
System.out.printf("Main: Waiting for the end of the three
    steps\n");
CompletableFuture<Void> waitFuture = CompletableFuture
    .allOf(step1Future, write2Future,
        step3Future);
```

12. Also, we execute a final step to write a message in the console:

```
CompletableFuture<Void> finalFuture = waitFuture
                                .thenAcceptAsync((param) -> {
                                    System.out.printf("Main: The CompletableFuture example has
                                                        been completed.");
                                });
finalFuture.join();
```

How it works...

We can use a `CompletableFuture` object with two main purposes:

- Wait for a value or an event that will be produced in future (creating an object and using the `complete()` and `get()` or `join()` methods).
- To organize a set of tasks to be executed in a determined order so one or more tasks won't start their execution until others have finished their execution.

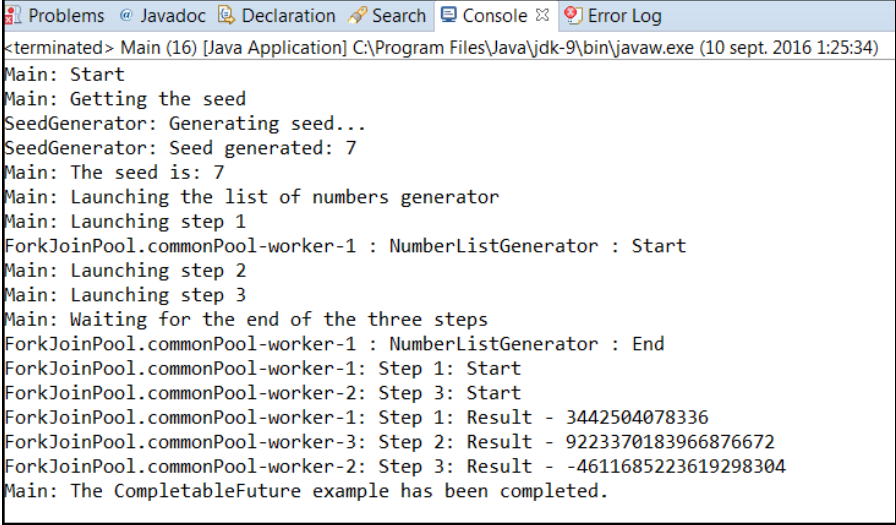
In this example, we made both uses of the `CompletableFuture` class. First, we created an instance of this class and sent it as a parameter to a `SeedGenerator` task. This task uses the `complete()` method to send the calculated value, and the `main()` method uses the `get()` method to obtain the value. The `get()` method sleeps the current thread until `CompletableFuture` has been completed.

Then, we used the `supplyAsync()` method to generate a `CompletableFuture` object. This method receives an implementation of the `Supplier` interface as a parameter. This interface provides the `get()` method that must return a value. The `supplyAsync()` method returns `CompletableFuture`, which will be completed when the `get()` method finishes its execution; the value of completion is the value returned by that method. The `CompletableFuture` object returned will be executed by a task in the `ForkJoinPool` returns the static method `commonPool()`.

Then, we used the `thenApplyAsync()` method to link some tasks. You call this method in a `CompletableFuture` object, and you must pass an implementation of the `Function` interface as a parameter that can be expressed directly in the code using a functional style or an independent object. One powerful characteristic is that the value generated by `CompletableFuture` will be passed as a parameter to the `Function`. That is to say, in our case, all the three steps will receive a random list of numbers as parameters. The `CompletableFuture` class returned will be executed by a task in the `ForkJoinPool` returns the static method `commonPool()`.

Finally, we used the `allOf()` static method of the `CompletableFuture` class to wait for the finalization of various tasks. This method receives a variable list of `CompletableFuture` objects and returns a `CompletableFuture` class that will be completed when all the `CompletableFuture` class passed as parameters are completed. We also used the `thenAcceptAsync()` method as another way to synchronize tasks because this method receives `Consumer` as a parameter that is executed by the default executor when the `CompletableFuture` object used to call the method is completed. Finally, we used the `join()` method to wait for the finalization of the last `CompletableFuture` object.

The following screenshot shows the execution of the example. You can see how the tasks are executed in the order we organized:



```
Problems @ Javadoc Declaration Search Console Error Log
<terminated> Main (16) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe (10 sept. 2016 1:25:34)
Main: Start
Main: Getting the seed
SeedGenerator: Generating seed...
SeedGenerator: Seed generated: 7
Main: The seed is: 7
Main: Launching the list of numbers generator
Main: Launching step 1
ForkJoinPool.commonPool-worker-1 : NumberListGenerator : Start
Main: Launching step 2
Main: Launching step 3
Main: Waiting for the end of the three steps
ForkJoinPool.commonPool-worker-1 : NumberListGenerator : End
ForkJoinPool.commonPool-worker-1: Step 1: Start
ForkJoinPool.commonPool-worker-2: Step 3: Start
ForkJoinPool.commonPool-worker-1: Step 1: Result - 3442504078336
ForkJoinPool.commonPool-worker-3: Step 2: Result - 9223370183966876672
ForkJoinPool.commonPool-worker-2: Step 3: Result - -4611685223619298304
Main: The CompletableFuture example has been completed.
```

There's more...

In the example of this recipe, we used the `complete()`, `get()`, `join()`, `supplyAsync()`, `thenApplyAsync()`, `thenAcceptAsync()`, and `allOf()` methods of the `CompletableFuture` class. However, this class has a lot of useful methods that help increase the power and flexibility of this class. These are the most interesting ones:

- **Methods to complete a `CompletableFuture` object:** In addition to the `complete()` method, the `CompletableFuture` class provides the following three methods:
 - `cancel()`: This completes `CompletableFuture` with a `CancellationException` exception.
 - `completeAsync()`: This completes `CompletableFuture` with the result of the `Supplier` object passed as a parameter. The `Supplier` object is executed in a different thread by the default executor.
 - `completeExceptionally()`: This method completes `CompletableFuture` with the exception passed as a parameter.
- **Methods to execute a task:** In addition to the `supplyAsync()` method, the `CompletableFuture` class provides the following method:
 - `runAsync()`: This is a static method of the `CompletableFuture` class that returns a `CompletableFuture` object. This object will be completed when the `Runnable` interface is passed as a parameter to finish its execution. It will be completed with a void result.
- **Methods to synchronize the execution of different tasks:** In addition to the `allOf()`, `thenAcceptAsync()`, and `thenApplyAsync()` methods, the `CompletableFuture` class provides the following methods to synchronize the execution of tasks:
 - `anyOf()`: This is a static method of the `CompletableFuture` class. It receives a list of `CompletableFuture` objects and returns a new `CompletableFuture` object. This object will be completed with the result of the first `CompletableFuture` parameter that is completed.

- `runAfterBothAsync()`: This method receives `CompletionStage` and `Runnable` objects as parameters and returns a new `CompletableFuture` object. When `CompletableFuture` (which does the calling) and `CompletionStage` (which is received as a parameter) are completed, the `Runnable` object is executed by the default executor and then the `CompletableFuture` object returned is completed.
- `runAfterEitherAsync()`: This method is similar to the previous one, but here, the `Runnable` interface is executed after one of the two (`CompletableFuture` or `CompletionStage`) are completed.
- `thenAcceptBothAsync()`: This method receives `CompletionStage` and `BiConsumer` objects as parameters and returns `CompletableFuture` as a parameter. When `CompletableFuture` (which does the calling) and `CompletionStage` (which is passed as a parameter), `BiConsumer` is executed by the default executor. It receives the results of the two `CompletionStage` objects as parameters but it won't return any result. When `BiConsumer` finishes its execution, the returned `CompletableFuture` class is completed without a result.
- `thenCombineAsync()`: This method receives a `CompletionStage` object and a `BiFunction` object as parameters and returns a new `CompletableFuture` object. When `CompletableFuture` (which does the calling) and `CompletionStage` (which is passed as a parameter) are completed, the `BiFunction` object is executed; it receives the completion values of both the objects and returns a new result that will be the completion value of the returned `CompletableFuture` class.
- `thenComposeAsync()`: This method is analogous to `thenApplyAsync()`, but it is useful when the supplied function returns `CompletableFuture` too.
- `thenRunAsync()`: This method is analogous to the `thenAcceptAsync()` method, but in this case, it receives a `Runnable` object as a parameter instead of a `Consumer` object.

- Methods to obtain the completion value: In addition to the `get()` and `join()` methods, the `CompletableFuture` object provides the following method to get the completion value:
 - `getNow()`: This receives a value of the same type of the completion value of `CompletableFuture`. If the object is completed, it returns the completion value. Else, it returns the value passed as the parameter.

See also...

- The *Creating a thread executor and controlling its rejected tasks* and *Executing tasks in an executor that returns a result* recipes in *Chapter 4, Thread Executors*